

How Portable Are LLM-Serving Scheduler Rankings Across Workloads, Operating Regions, and Metrics?

Soroush Vahidi

Draft V1 — pre-Phase-12 authoritative consolidation, 2026-09-02

Abstract

Large language model (LLM) serving systems rely on sophisticated scheduling mechanisms, evaluated on workloads that are themselves heterogeneous across source, time, and operating region. We ask a second-order question: is the *comparative ranking* a scheduler benchmark reports portable, formalized as $R(W, M, L)$, a comparative scheduler ranking as W (workload source/window), M (metric), and L (operating/load region) vary? We present the LLM-Serving Scheduler Portability Benchmark (LSSP): a preregistered, paired evaluation spanning three workload sources, 120 workload windows, a calibrated six-region operating grid, an 11-primary/13-total scheduling-policy panel, and 18,720 completed cells, plus a benchmark sample-complexity study and a targeted real-vLLM validation design. We establish, as evidence available now, that the three workload sources are measurably different and strongly separable in their observed descriptor distributions, and, as historical pilot evidence, that scheduler differentiation is real but strongly source-dependent. LSSP’s central results remain outstanding and are stated as explicit, outcome-neutral result contracts.

Keywords: LLM inference serving; request scheduling; ranking portability; performance evaluation; workload characterization; benchmark reproducibility

Contents

1	Introduction	3
2	Related Work	5
2.1	Serving Architectures and Scheduling Mechanisms	5
2.2	Production Workloads and Characterization	6
2.3	Simulators and Benchmark Frameworks	6
2.4	Workload Dependence and Benchmark Sensitivity	7
2.5	Ranking Portability and Benchmark Methodology	7
2.6	Recent <i>Journal of Supercomputing</i> Context	7
2.7	Position of LSSP	8
3	Study Design	8
3.1	Research Questions	8
3.2	Study Overview	9
3.3	Evidence Independence / Publication Boundaries	9
3.4	Experimental Unit and Paired Design	10
3.5	Preregistration	10

4	Workload Corpus and Characterization	10
4.1	Sources	10
4.2	Representation and Provenance	10
4.3	Window Construction	11
4.4	Descriptors	11
4.5	Cross-Source Differences	11
4.6	Source Separability	11
4.7	Drivers of Differences	12
4.8	Window-Size Sensitivity	12
5	Scheduler Benchmark Methodology	12
5.1	Scheduler Panel	12
5.2	Fidelity Taxonomy	13
5.3	Execution Model	13
5.4	Stage-0 Discriminability Pilot (Historical)	13
5.5	Operating-Region Calibration	14
5.6	Metrics	14
5.7	Paired Design and Repetitions	14
5.8	Mechanism Telemetry	15
5.9	Ranking-Uncertainty Quantification and Reversal Classification	15
5.10	Temporal Analysis Design	16
5.11	Robustness Analyses	16
5.12	SLO-Definition Sensitivity: A Disclosed Protocol Gap	16
6	Scheduler Ranking Portability	17
6.1	Discriminability	17
6.2	Cross-Source Rankings	17
6.3	Pairwise Reversals	17
6.4	Temporal/OOD Portability	18
6.5	Load Conditioning	18
6.6	Metric Conditioning	18
6.7	Robustness	18
7	Benchmark Sample Complexity	18
7.1	Motivation	18
7.2	Design	19
8	Real-System Validation	19
8.1	Objective	19
8.2	Planned Design	19
8.3	Result Placeholder	20
9	Discussion	21
9.1	If Ranking Portability Is High	21
9.2	If Ranking Portability Is Low	21
9.3	Mechanism Equivalence and Collapse	21
9.4	Workload Descriptor Interpretation	21
9.5	Benchmark Construction and Practical Reproducibility	22

10 Threats to Validity	22
11 Reproducibility and the LSSP Artifact	23
12 Conclusion	24
Declarations	24

1 Introduction

Modern LLM serving targets high-performance, GPU-resident inference systems whose scheduling decisions govern how requests share costly accelerator, memory, and batching resources, and serving stacks have grown increasingly sophisticated in how they make those decisions. Beyond simple first-come-first-served execution, modern serving stacks employ continuous batching and paged key-value (KV) cache management [10], iteration-level scheduling [27], chunked prefill to balance prefill and decode work within a batch [1], disaggregated prefill/decode execution across dedicated hardware pools [11, 31], KV-cache-centric disaggregated architectures [12], dynamic instance-level rebalancing [19], explicit fairness objectives [18], output-length-aware admission and memory packing [9], and learning-to-rank decode-priority policies [8]. Each of these mechanisms was introduced, and evaluated, against a particular choice of workload: a specific trace, a specific arrival process, a specific mix of prompt and output lengths, and a specific operating load.

A separate line of work has documented that these workloads are themselves far from uniform. Public production traces from different services and time periods differ substantially in burstiness, request concurrency, prompt and output length distributions, and failure characteristics [11, 21, 24]; large-scale characterization studies of KV-cache reuse and of year-long production traffic report further axes of heterogeneity across clients, models, and time [3, 17]; and dedicated workload-generation frameworks exist precisely because no single fixed trace is thought to be representative of production traffic in general [26]. Individually, prior work has therefore already established two things: that LLM-serving mechanisms are numerous and mechanistically diverse, and that the workloads used to evaluate them are themselves heterogeneous and shift across source, time, and operating region.

What has not been directly asked is a second-order question that follows from combining these two observations. The typical evaluation question in this literature is: *how well does scheduler A perform, relative to scheduler B, on the workload(s) chosen for this paper’s experiments?* Given that workloads are heterogeneous, and that a paper’s experiments necessarily use only a finite, chosen set of workloads, a comparative claim of this form is only as useful as its *portability*: does the same comparative ordering of schedulers hold if the workload distribution, the operating load, or the evaluation metric is changed to another one a practitioner might reasonably care about? This is a different question from workload heterogeneity itself, and a different question from any single scheduler’s sensitivity to load or SLO. It is a question about the reliability of the *comparison*, not about any individual system.

We formalize this as the portability of a comparative scheduler ranking $R(W, M, L)$, where W denotes a workload distribution, window, or source; M denotes an evaluation metric; and L denotes an operating or load region. $R(W, M, L)$ is the ordering (full or partial) that a fixed panel of scheduling policies induces under a fixed benchmark protocol, holding W , M , and L at specified values. Ranking portability asks how R behaves as W , M , or L is varied: whether the ordering of policies is preserved, where and how it reverses, and how much evidence (how many independent

workload windows, drawn from how many sources) is required before a reported ranking can be trusted to generalize beyond the exact windows used to produce it.

This question matters because a scheduler comparison is scientifically and practically useful only insofar as its comparative conclusion transfers to the workload distributions a reader actually cares about. A benchmark result of the form “scheduler A outperforms scheduler B ” is not a claim about A and B in the abstract; it is implicitly a claim about A and B *under the workload, metric, and load conditions the benchmark used*. If that comparative ordering is fragile under source, metric, or load-region substitution, then benchmark results risk being read as more general than they are, and evaluation effort risks being spent on a single workload family when the resulting conclusion may not transfer. Conversely, if comparative orderings are portable across independent sources, metrics, and load regions, that is itself a useful, non-obvious finding: it would mean a leaner benchmark protocol is sufficient for future scheduler comparisons, and it would give scheduler designers a much stronger warrant for claims of general improvement.

We refer to the benchmark built around this question as the LLM-Serving Scheduler Portability Benchmark (LSSP). LSSP asks the following research questions, fixed under a preregistered protocol before any comparative outcome was generated (§3):

- RQ1.** How stable are scheduler rankings across independent workload sources?
- RQ2.** How stable are scheduler rankings under temporal, provider, and domain shifts?
- RQ3.** To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?
- RQ4.** Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?
- RQ5.** How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?
- RQ6.** Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Load-level dependence is deliberately treated as a secondary robustness/sensitivity axis applied to RQ1–RQ4, rather than as a headline research question in its own right, to avoid duplicating a closely related prior load-scaling study on an overlapping window set (§3).

Contributions. Because LSSP’s comparative scheduler outcomes had not yet been generated at the time this manuscript was prepared (§3), we state our contributions in a form that does not depend on, and is not falsified by, either possible direction of that future result:

- We formulate the portability of a comparative scheduler ranking, $R(W, M, L)$, as a benchmark object in its own right, distinct from both workload characterization and single-scheduler evaluation.
- We design a paired, multi-source evaluation protocol that holds a fixed panel of scheduling policies constant across independently sourced workload windows, operating regions, and metrics, so that ranking-portability statistics are computed on genuinely paired observations rather than on separately tuned, per-source experiments.

- We assemble a mechanism-diverse, fixed scheduler panel spanning classical, fairness-aware, KV-aware, deadline-aware, and faithfully reimplemented external policies, selected under an outcome-independent selection rule fixed before any pilot result was observed (§5).
- We define an uncertainty and reversal-detection methodology that distinguishes practically meaningful rank reversals from microscopic sign changes, using preregistered practical-margin and confidence-interval criteria, and that treats every completion-conditioned metric’s undefined cases explicitly rather than by imputation (§5).
- We define a benchmark sample-complexity methodology that asks how many independent workload windows are required before a reported ranking is likely to reproduce, rather than assuming a conventionally sized window set is sufficient.
- We characterize the workload corpus underlying the benchmark, establishing that the workload sources used are measurably different in observed descriptor distributions and strongly separable under a leakage-resistant, chronologically blocked evaluation (§4).
- We plan a paired public artifact release of the benchmark’s frozen workload identities, policy panel, protocol, and (once generated) outcome and analysis artifacts, to support independent reproduction (§11).

Consistent with the scientific state of the project at the time of writing (§3), this manuscript does not claim that scheduler rankings are stable, that they are unstable, that any reversal exists, or that any scheduler in the panel wins. Sections 6 through 8 state the exact, preregistered result that each pending experiment will report, without asserting its direction in advance.

2 Related Work

2.1 Serving Architectures and Scheduling Mechanisms

A substantial body of systems work has improved LLM inference throughput and latency through scheduling and memory-management mechanisms. Orca introduced iteration-level scheduling, interleaving requests at the granularity of individual decoding steps rather than whole requests [27]. vLLM introduced PagedAttention, a paged virtual-memory-style KV-cache allocator that substantially raised achievable batch sizes and became a widely adopted serving-system reference point [10]. Sarathi-Serve introduced chunked prefill, splitting long prefill computations across multiple decode-interleaved steps to reduce head-of-line blocking under continuous batching [1]. Splitwise and DistServe independently proposed disaggregating the compute-bound prefill phase and the memory-bound decode phase onto separate hardware pools connected by fast interconnects, each showing throughput or cost improvements from phase-specific resource allocation [11, 31]; Mooncake extends this direction with a KV-cache-centric disaggregated architecture that treats a shared, elastic KV cache – rather than the compute engines themselves – as the central resource to schedule around [12]. Llmunix adds dynamic, cross-instance request migration and rebalancing on top of a serving cluster [19]. DynamoLLM addresses cluster-level energy and cost efficiency by dynamically reconfiguring an inference cluster’s resource allocation under latency SLOs [21].

A second cluster of work targets scheduling *policy* design more specifically. VTC (Virtual Token Counter) defines an explicit token-level fairness objective for continuous-batching schedulers and proves a bound on the resulting service-difference guarantee [18]. S^3 predicts each request’s output length ahead of generation and uses that prediction to pack KV-cache memory more tightly

at admission time, trading occasional misprediction-driven preemption for higher achievable batch size [9]. Fu et al. [8] instead learns a pairwise ranking model over waiting requests to prioritize decode order – a distinct mechanism from S^3 ’s admission-time length prediction, despite both papers appearing at NeurIPS in consecutive years and both targeting throughput under uncertain output length. FastServe, JITServe, PARS, Libra, SLAI/RAD, SCORPIO, LMetric, Apt-Serve, and ProServe extend this space further with iteration-level preemption, SLO-aware admission under imprecise request information, pairwise learning-to-rank variants, dynamic request partitioning, theoretically motivated multi-objective scheduling, heterogeneous-SLO-aware admission, lightweight scheduling metrics, adaptive hybrid-cache scheduling, and unified multi-priority scheduling, respectively [4, 13, 14, 16, 22, 23, 25, 29, 30]. Each of these papers reports an improvement over one or more baselines on the workload(s) chosen for its own experiments; none of them, individually or collectively, asks whether the *relative ordering* they report would be preserved under an independently sourced workload, a different metric, or a different operating load – the question this benchmark is built around (§2.7).

2.2 Production Workloads and Characterization

A parallel literature documents that LLM-serving workloads observed in production are themselves heterogeneous. BurstGPT releases 10.31 million traces collected from a regional deployment of Microsoft’s Azure OpenAI GPT service over 213 days, characterizing request burstiness, conversation structure, and response-length behavior, and is now published as a full KDD 2025 paper rather than only as a preprint [24]. Two official Microsoft releases document Azure’s own internal LLM-inference traffic: Splitwise analyzes production conversation and code-completion traces collected on Azure in November 2023 [11], and DynamoLLM analyzes a longer, one-week Azure trace collected in May 2024 [21]. anonymized per venue record; see primary source [3] characterizes KV-cache reuse patterns at a large cloud provider, finding that reuse probability and lifespan are workload-category-dependent in ways that synthetic workload studies had not captured. see primary source for full author list [17] reports a one-year longitudinal characterization of production traffic spanning workload evolution, caching behavior, and load balancing (a recent preprint at the time of writing; §2.7). ServeGen instead characterizes a worldwide production inference service in order to build a principled per-client workload *generator*, motivated explicitly by the position that no single fixed trace is representative enough on its own [26]; because it characterizes the same underlying Alibaba Cloud platform as this project’s Bailian/Qwen source, we do not count it as an independent workload provider (§4). TraceLab characterizes a distinct workload family entirely – coding-agent sessions rather than single-turn or short-multi-turn chat requests – and is used here only as out-of-distribution context, not as one of this project’s primary Pilot-V2 sources.

2.3 Simulators and Benchmark Frameworks

Vidur is the closest existing infrastructure precedent to LSSP: a large-scale simulation framework purpose-built for exploring LLM-inference system configurations at a fidelity and speed unavailable from running full experiments on real hardware [2]. LLMServingSim and its successor LLMServingSim 2.0 provide a related but architecturally different hardware/software co-simulation infrastructure, adding iteration-granularity simulation with algorithmic-redundancy reuse and, in the 2.0 revision, explicit support for heterogeneous accelerators and disaggregated serving techniques [6, 15]. These simulators are built to let researchers explore serving-system design points cheaply; none of them is built around, or reports, a benchmark of *comparative scheduler ranking portability* across independently sourced workloads in the sense this paper defines. LSSP’s own simulator

(§5) is project-local infrastructure, not a contribution positioned against this literature; we make no claim that its output is equivalent to production hardware latency or throughput, only that it supports a fixed, controlled benchmark protocol for relative comparisons (§5).

2.4 Workload Dependence and Benchmark Sensitivity

Prior work has already established, individually, that LLM-serving workloads are heterogeneous (§2.2) and that individual serving mechanisms and schedulers are sensitive to batching structure, prefill/decode balance, load level, and service-level objectives (§2.1). We treat both of these as accepted background, not as a novel contribution of this work. What we do not find in the literature reviewed here is a direct study of whether a *comparative ordering* among a fixed panel of schedulers is itself portable across independently sourced workloads, operating regions, and metrics – the distinction this paper’s Introduction develops between single-system sensitivity and comparative-ranking portability.

2.5 Ranking Portability and Benchmark Methodology

The statistical methodology this benchmark uses – Kendall’s tau-b and Spearman’s rho for pair-wise ranking agreement, paired comparison and block-bootstrap confidence intervals for ranking-uncertainty quantification, and a probability-of-correct-selection framing for benchmark sample complexity – draws on standard nonparametric statistics and comparative-algorithm-ranking methodology (§5); these are general statistical tools, not literature claims specific to LLM serving, and are cited here only as the methods this benchmark’s analysis plan applies to its own, independently generated data. The closest methodological precedent we identified is from the classical parallel-job- scheduling literature: Zakay and Feitelson [28] partition a real scheduler workload trace into independent per-user subtraces and recombine them to resample varied-but-realistic workloads, evaluating how a *single* parallel job scheduler’s measured performance responds to workload resampling. LSSP’s sample-complexity question (§7) is the same evaluation instinct – using resampling to ask how much workload evidence a scheduler-performance conclusion actually rests on – applied to a different object: the portability of a *comparative* ranking among multiple schedulers across independently sourced workloads, rather than one scheduler’s sensitivity to resampled variants of a single trace.

2.6 Recent *Journal of Supercomputing* Context

Three recent *Journal of Supercomputing* papers illustrate the journal’s current engagement with GPU-resident LLM-inference performance and portability, at layers adjacent to but distinct from this benchmark’s request-scheduling focus. Doosthosseini et al. [7] address deploying LLM inference services on Slurm-managed HPC clusters without compromising existing cluster security policy – a *cluster job-placement* scheduling problem, one level above the *intra-server request* scheduling this benchmark studies. Carmona-Martínez et al. [5] and Siwinska et al. [20] each study a different sense of “portability” for LLM/transformer workloads on GPU (and, in the latter case, TPU) hardware: compiler-backend selection trade-offs and auto-tuning-framework portability across accelerator hardware, respectively – portability of *implementation performance across hardware*, not portability of a *comparative ranking across workloads* at fixed hardware, which is this benchmark’s object of study. We cite these three papers to situate LSSP within the journal’s live interest in LLM-serving and GPU-inference performance, not because they address this benchmark’s specific research question.

2.7 Position of LSSP

LSSP asks a different question than the majority of the prior work surveyed above. Prior work generally studies a scheduler, a serving system, a simulation environment, or a workload generator/characterization in its own right. LSSP instead studies the portability of *comparative* scheduler rankings across independently sourced workload families, operating regimes, and metrics, using a fixed, mechanism-diverse scheduler panel and a preregistered protocol. To our knowledge, the LSSP benchmark is distinct in treating the portability of comparative scheduler rankings itself as the primary object of study across independent workload sources, operating regimes, and metrics, while explicitly quantifying ranking uncertainty and benchmark sample complexity. A literature search targeting exactly this question (scheduler-ranking stability/portability, cross-workload scheduler evaluation, benchmark sample complexity and representativeness for LLM-serving systems, current through September 2026) did not surface prior work that treats comparative scheduler-ranking portability itself as its primary object across independent LLM-serving workload sources; the closest single-scheduler-focused methodological precedent (Zakay and Feitelson [28], §2.5) predates the LLM-serving literature entirely and evaluates one scheduler’s sensitivity to workload resampling, not comparative ranking across schedulers. Vidur is the closest infrastructure precedent (§2.3); LLMservingSim and its successor are related but differently scoped simulator infrastructure; the recent production-characterization literature (§2.2) establishes the workload heterogeneity this benchmark’s design responds to, but does not itself study cross-source ranking portability. This wording is intentionally narrower than an unconditional claim of priority; we do not claim to be the first to study LLM-serving workload dependence or scheduler sensitivity, only that we did not identify a prior benchmark treating cross-source, cross-metric, cross-load-region scheduler-ranking portability as its primary object of study under the primary-source review conducted for this manuscript.

A note on provider independence is warranted here because it bears directly on how several of the sources above should be read together (§4 gives the full treatment). BurstGPT’s own reported methodology collects its traces from a regional customer’s use of Microsoft’s Azure OpenAI GPT service [24]; this project’s own Azure-sourced traces are Microsoft’s own official releases of internal Azure LLM-inference-service traffic [11, 21]. Both therefore sit on the same underlying cloud provider’s infrastructure, even though they are collected by different teams through different mechanisms and released as distinct, independently versioned artifacts. We describe BurstGPT and this project’s Azure traces as distinct, independently released workload sources throughout this manuscript, and we do not describe them as independent providers.

3 Study Design

3.1 Research Questions

We reproduce the project’s frozen research questions verbatim (see `docs/RESEARCH_QUESTIONS.md` for the full changelog):

RQ1. How stable are scheduler rankings across independent workload sources?

RQ2. How stable are scheduler rankings under temporal, provider, and domain shifts?

RQ3. To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?

- RQ4.** Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?
- RQ5.** How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?
- RQ6.** Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Load-level dependence is a secondary robustness/sensitivity analysis applied to RQ1–RQ4’s findings, not a headline research question (§3.3).

3.2 Study Overview

LSSP evaluates a fixed panel of 13 scheduling policies (§5.1) against a frozen corpus of 120 workload windows drawn from three independently sourced traces (§4.1), across a calibrated six-region grid of operating regions (§5.5) and a fixed set of primary and completion-conditioned metrics (§5.6). Every policy is run against every window under every operating region, with a deterministic rep0/rep1 verification pass (§5.7), producing $120 \times 6 \times 13 \times 2 = 18,720$ completed, independently validated cells, from which per-source, per-region, and per-metric scheduler rankings are derived. Ranking-portability statistics (Kendall’s tau-b, Spearman’s rho, top- k overlap, and pairwise reversal detection) are then computed across pairs of sources, regions, and metrics, using workload window as the statistical unit (§3.4). This design was frozen (§3.5) before any comparative scheduler outcome existed.

3.3 Evidence Independence / Publication Boundaries

This project shares a lineage and a simulator codebase with two other, separately scoped manuscripts, and its design was explicitly audited for overlap against both before this study’s protocol was frozen (docs/OVERLAP_LEDGER.md, docs/GO_NO_GO_GATES.md Gate A, docs/EVIDENCE_INDEPENDENCE_PLAN.md):

- **“The Exploitability Gap in LLM-Serving Scheduler Portfolios” (LLM 2026).** That manuscript’s `public_replay_load_scaling_v1/v2` experiment used a fixed set of 60 canonical public-trace windows (20 BurstGPT + 20 Azure-2023 conversation + 20 Azure-2023 code), a load-factor grid $\{1, 2, 4, 8, 16, 32, 64, 128\}$, an 8-policy portfolio, and ANWG/SBS-VBS exploitability diagnostics. That result is treated here as `PRIOR_RESULT_REFERENCE_ONLY`: citable as prior motivation, never restated as this project’s own evidence. This project’s own window sampling is independently constructed (§4.3) and verified non-identical to that 60-window set; load-level dependence is accordingly demoted to a secondary analysis (§3.1) using this project’s own calibration protocol (§5.5), never that load grid.
- **“Module Counterfactuals for Attributing LLM Serving Scheduler Effects” (a separate module-intervention manuscript).** This study makes no module-counterfactual or module-attribution claim; RQ4’s workload-descriptor analysis (§5) is an offline, descriptive association analysis, not a deployed selector or an intervention-based attribution method.
- Other project lineage (an unrelated database/systems research thread) is unconnected to this manuscript’s content and is not referenced here.

3.4 Experimental Unit and Paired Design

The statistical unit of analysis is the workload window: a fixed-size (200-request) contiguous slice of a source trace, uniquely identified and frozen before any scheduler outcome existed (§4.3). Every policy in the panel (§5.1) is run against every window under every calibrated operating region (§5.5), so that ranking-portability statistics are computed on genuinely paired observations – the same window, region, and metric definition, varying only the policy – rather than on separately tuned per-source experiments. Source, operating region, and metric are treated as factors of interest; policy identity is the object whose relative ordering is measured.

3.5 Preregistration

The workload window set (120 windows: 40 per source, 10 Stage-0-reused + 30 Pilot-V2-new per source; §4.3), the scheduler panel and its selection rule (§5.1), the metric contract (§5.6), the six-region operating-load calibration (§5.5), and the ranking-portability analysis plan (reversal-detection thresholds, bootstrap procedure, sample-complexity design) were each frozen, and cross-checked against a structural validation gate, before any Pilot-V2 comparative scheduler outcome was generated. `docs/RANKING_PORTABILITY_PILOT_V2_PROTOCOL.md`, `docs/RANKING_PORTABILITY_ANALYSIS_PLAN.md`, and `docs/RANKING_PORTABILITY_WINDOW_FREEZE.md` record the exact frozen content and the gate checks each passed.

4 Workload Corpus and Characterization

4.1 Sources

LSSP’s primary Pilot-V2 corpus draws from three independently released workload sources: BurstGPT [24], Microsoft’s official Azure 2024 LLM-inference trace [21], and a Bailian/Qwen production trace release. Azure’s 2023 conversation/code traces [11] and Stage-0’s historical windows contribute additional context but are not part of the primary Pilot-V2 120-window matrix beyond the 10-per-source Stage-0-reused windows described below. TraceLab [32] is retained as an out-of-distribution, coding-agent workload family for future expansion, not as one of the three primary sources, because its independence from a prior 512-window public sweep cannot be fully verified from public artifacts alone (`docs/TRACELAB_PROVENANCE_RESOLUTION.md`).

As established in §2.7, BurstGPT’s own reported collection methodology logs traffic from a regional customer of Microsoft’s Azure OpenAI GPT service, and this project’s Azure trace is Microsoft’s own official release of internal Azure LLM-inference-service traffic. Both therefore sit on the same underlying cloud provider’s infrastructure. We describe BurstGPT and Azure as distinct, independently released workload sources – different collection methodology, different customer population, different release provenance and version history – and we do not describe them as independent providers. Bailian/Qwen (Alibaba Cloud) is a genuinely different underlying platform from both.

4.2 Representation and Provenance

Each source is ingested through a dedicated, field-level-audited adapter (`docs/DATA_FIELD_PROVENANCE.md`) that maps the source’s native schema onto a common internal request representation (arrival time, input tokens, output tokens, total tokens, session identifier where available, model class). No field is fabricated where a source’s native schema lacks it (for example, BurstGPT’s documented schema has no tenant, KV-reuse, SLO, or failure-code field, and none is synthesized).

License status for each source is recorded in `docs/DATA_LICENSE_AUDIT.md` (BurstGPT: CC-BY-4.0) and re-verified before any confirmatory use of a real, full-scale sample.

4.3 Window Construction

The Pilot-V2 corpus freezes exactly 120 windows of 200 requests each: 40 per source (`azure_llm_2024`, `bailian_qwen`, `burstgpt`), each composed of 10 windows reused verbatim from the Stage-0 pilot (§5.4) plus 30 new windows drawn by a deterministic, outcome-blind, source-symmetric pinned-extension sampler that uses only valid-row index ranges and fixed seeds – never scheduler metrics, policy identities, or Stage-0 tie/non-tie outcomes (`BURSTGPT_OUTCOME_TARGETING = NO`, structurally verified). Windows are assigned to EARLY/MIDDLE/LATE chronology strata by deterministic relative chronology (60/30/30 windows respectively); Stage-0-reused windows are all EARLY. The freeze passed a full structural validation gate – unique window identities, exact per-source and Stage-0/new counts, fixed `request_count = 200`, no prohibited overlap, valid chronology, and absence of any scheduler/policy/load-calibration outcome field in the selection rule – recorded with a canonical content hash (excluding generation timestamp; full value in Table 1) before any Pilot-V2 load calibration or scheduler outcome was generated.

4.4 Descriptors

Each window is described by a 28-feature descriptor vector spanning prompt- and output-token-length statistics (mean, p90, long-prompt fraction at the 512/2048/8192-token thresholds), arrival-timing statistics (mean arrival rate, inter-arrival percentiles), and derived load-pressure proxies. Three deterministically redundant pressure-proxy features ($R^2 = 1.0$ against other included features) are retained in the underlying dataset but excluded from the classifier input used in §4.6. Prompt-token figures compose differently across sources for architectural reasons: TraceLab’s and Azure’s prompt-token fields are potentially multi-turn-cumulative context, while BurstGPT’s is a single per-request figure. This is a property of how each source’s underlying workload is architected (a growing agent/conversation session versus a single stateless API call), not an adapter artifact, and it is preserved as an explicit caveat on every prompt-length-based claim below.

4.5 Cross-Source Differences

Under a leave-one-time-bucket-out (EARLY/MIDDLE/LATE), grouped, chronologically blocked cross-validation over $n = 400$ windows (window size 200), a random forest classifier distinguishes the four descriptor-labeled source classes with pooled balanced accuracy 1.000 (mean $\pm$ std across three folds: 1.000 ± 0.000); a multinomial logistic regression reaches 0.988 balanced accuracy (0.987 ± 0.007 across folds); and a depth-limited (depth ≤ 3) decision tree reaches 0.980 balanced accuracy (0.980 ± 0.018 across folds, worst single fold 0.955 with LATE held out). This grouped, chronologically blocked evaluation supersedes an earlier random-split result (which agreed at balanced accuracy 1.0) for any manuscript claim, because random splitting is vulnerable to within-window-adjacent-time leakage that the grouped evaluation is structurally immune to (regression tested; `tests/test_separability_pipeline_no_leakage.py`).

4.6 Source Separability

We report the final characterization result as `SEPARABILITY_RESULT_CONFIRMED_WITH_CAVEATS`. The safe claim supported by this evidence is that the workload sources’ observed descriptor distributions are measurably different, and that source identity is strongly separable under a leakage-

resistant, chronologically blocked evaluation. This is not evidence that the underlying providers’ conversations or tasks intrinsically differ in length or structure, only that their *observed, request-level* descriptor figures do, for a mixture of architectural and possibly-genuine-content reasons that this classifier-based analysis cannot separate (§4.4).

4.7 Drivers of Differences

An attribution analysis triangulating held-out permutation importance, single-feature balanced accuracy, leave-one-feature-out deltas, and feature-group-ablation deltas (all under the same grouped chronological cross-validation) identifies two tiers of driving features. Tier 1 – incrementally load-bearing, with nonzero permutation importance and a measurable accuracy cost when the `prompt_length` feature group is removed (-0.0051) – consists of `long_prompt_fraction_512`, `long_prompt_fraction_2048`, and `prompt_tokens_mean`. Tier 2 – individually strong single-feature classifiers (balanced accuracy up to 0.980) whose removal, individually or as a whole feature group, costs 0.000 accuracy because their signal is redundant with Tier 1 – includes total/prompt-token percentile features, inter-arrival percentiles, and mean arrival rate. Prompt-length structure, specifically the fraction of long prompts at the 512- and 2048-token thresholds and mean prompt length, is therefore the primary, non-redundant driver of source separability; arrival-timing structure and output length are individually predictive but carry no additional signal once prompt-length features are present. A critical-ablation check confirms this separability is broad rather than dependent on any single feature: removing the two strongest Tier-1 features together changes random-forest balanced accuracy by at most -0.0025 .

4.8 Window-Size Sensitivity

The Pilot-V2 corpus uses a fixed window size of 200 requests throughout, matching Stage-0 and the LLM 2026 comparison window size for methodological consistency. A dedicated window-size sensitivity sweep (varying window size and re-checking separability/discriminability) has not yet been run; this is noted as an open item in §10 rather than a result reported here.

5 Scheduler Benchmark Methodology

5.1 Scheduler Panel

The PRIMARY panel comprises 11 policies, executed alongside 2 additional `STYLE_APPROXIMATION` policies used only for a mandatory robustness check (13 policies executed in total): a classical arrival-order baseline (`fifo`, also the load-calibration reference policy), a deadline-aware policy (`edf`), a slack/urgency-aware policy (`least_laxity_first`), a service-length-aware policy (`estimated_service_time_first`), a fairness-aware policy (`weighted_fair_share`), a KV/memory-pressure-aware policy (`kv_constrained_online`), an admission-control/SLO-guard policy (`admission_control`), and faithful reimplementations of four external mechanisms: vLLM-style token-budget continuous batching in both its pre-chunked-prefill and chunked-prefill forms (`vllm_faithful`, `vllm_chunked_prefill_faithful`), Sarathi-style chunked prefill (`sarathi_faithful`), and an SLO-aware scheduling policy in the SLAI/RAD family (`slai_faithful`). The two `STYLE_APPROXIMATION` robustness policies are a token-budget-proxy style approximation (`vllm_style_token_budget`) and an SLO-guard-proxy style approximation (`scorpio_style_slo_guard`); two further faithful reimplementations requiring disaggregated/cross-instance simulator semantics (`distserve_faithful`, `llumnix_faithful`) are executed only in a secondary

stratum, never pooled with the single-GPU colocated PRIMARY leaderboard, and one candidate (`apt_serve_faithful`) is excluded entirely as `scaffolding_only` – not a validated reimplement ation at the time of this freeze.

A policy enters the PRIMARY panel only if, at the time it was added to the comparability audit (frozen the day before Stage 0 executed), it satisfies an outcome-independent selection rule: its mechanism predates this protocol’s freeze, it occupies a mechanism family not already saturated by an existing panel member, it has adequate implementation fidelity and passes its own unit tests, it is simulator-semantically compatible with the single-GPU colocated execution model, and it is independently reproducible from a cited reference. No policy was added to or removed from the panel in response to any Stage-0 outcome (§5.4); the panel document predates Stage-0 execution by a full day, and mechanism families identified as under-implemented (preemptive/multilevel scheduling in the FastServe sense, JITServe, LMetric) were deliberately left as documented future-work candidates rather than fast-tracked into the panel after seeing Stage-0’s BurstGPT collapse.

5.2 Fidelity Taxonomy

Every executed policy is labeled with one of five fidelity classes: `FAITHFUL_EXTERNAL` (a validated reimplement ation of a specific published mechanism), `REPOSITORY_NATIVE_CLASSICAL` (a standard scheduling discipline implemented natively in this simulator, not claimed as a reproduction of any specific paper), `SIMULATOR_PROXY` (a mechanism-motivated policy without a single canonical external paper), `STYLE_APPROXIMATION` (an approximate, style-level proxy for a published mechanism, explicitly excluded from the PRIMARY headline panel and used only for robustness checks), and `scaffolding_only` (an unaudited, not-yet-validated implementation, excluded entirely). We do not claim that any `STYLE_APPROXIMATION` or `SIMULATOR_PROXY` policy is an official or validated reimplement ation of its namesake published system.

5.3 Execution Model

Each (window, policy, operating-region) cell is executed as a fully deterministic discrete-event simulation: given identical inputs, the simulator produces bit-identical outputs, with no consumed randomness (verified by a repeated-run determinism test). This determinism is what permits the paired design of §3.4: two policies’ outcomes on the same window and region differ only because of the policy’s own scheduling decisions, not because of independent sampling noise.

5.4 Stage-0 Discriminability Pilot (Historical)

Before the Pilot-V2 protocol described in the remainder of this section was frozen, a preregistered discriminability pilot (“Stage 0”) executed a real, 1,080-cell matrix (array 1213964; three sources, three load regions, a six-policy PRIMARY subset) to check, before committing to the full protocol, that the panel would yield non-trivial scheduler differentiation under a fixed load protocol without post-hoc cherry-picking. **This is historical pilot evidence, not the confirmatory Pilot-V2 result**, and it never measured cross-source ranking portability – that question was and remains explicitly out of Stage-0’s scope. Stage 0’s own five-criterion discriminability analyzer returned four passing criteria (77.8% of (source, window, load-region) triples non-tied, 86.7% of windows non-tied, no universal policy collapse, and 100% of non-tied cells showing meaningful metric variation) and one genuine failure: BurstGPT contributed only 14.3% of non-tied conditions, against a 15% floor, versus ~42.9% each for the other two sources – a result reported as `STAGE0_NO_GO`, robust to the completion-conditioned-metric zero-completion convention used. A follow-up diagnostic traced this to a mechanism-level explanation: differentiation was strongly source-dependent, attributable

to a near-total collapse of five of the six Stage-0 policies into pairwise bit-identical outcomes on BurstGPT’s structurally short-prompt, near-constant-output traffic – not to BurstGPT receiving weaker or fewer calibrated load conditions than the other sources. This motivated the Pilot-V2 protocol’s larger, mechanism-diverse 13-policy panel and denser six-region load grid (§5.5), applied symmetrically to every source, never to BurstGPT alone. Detailed Stage-0 mechanism analysis is deferred to an appendix; it is not restated here as a ranking-portability finding.

5.5 Operating-Region Calibration

A FIFO-only calibration pass establishes six operating regions – `LOW` (load factor 0.5), `PRE_KNEE` (0.8), `KNEE` (1.0), `POST_KNEE` (1.1), `OVERLOAD` (1.2), and `HIGH_PRESSURE` (1.5) – across all 120 frozen windows (720 calibration cells total; 720 valid, deterministic, 0 failed/missing/ duplicated). We report this as calibration provenance, not a comparative scheduler result: FIFO is the calibration reference policy, and the calibration’s role is to fix load-factor-to-region mappings before any other policy is executed against them, not to characterize FIFO’s relative performance. As a calibration characteristic worth preserving, 301 of the 720 calibration-region records (93 BurstGPT, 101 Bailian/Qwen, 107 Azure) recorded zero completions under FIFO at that region’s load factor; this is a property of the calibration design at those specific (window, region) combinations, the frozen calibration validity gate passed regardless (`PHASE11_CALIBRATION_VALID = YES`), completion-conditioned metrics are simply undefined (not zero) in those records (§5.6), and no calibration redesign was performed after observing this count.

5.6 Metrics

Three metrics are always defined for every completed cell: completion fraction, arrival-normalized weighted goodput (ANWG, the primary metric), and weighted completion fraction. A second group of metrics is defined only conditional on at least one request completing: SLO-violation rate, weighted goodput, mean/p95/p99 latency, and request/token throughput are each `NaN` if and only if completion fraction is exactly zero for that cell. Mean/p95 time-to-first-token is subject to a stricter, separately checked precondition (at least one completed request recorded a first-token time), because TTFT can be undefined even when completion fraction is positive. No undefined conditional-metric value is ever imputed as 0 or 1, in either direction, and never after inspecting which policy or source it would affect. Where a policy’s value is undefined for a given (window, region, metric) condition, that policy is excluded from the ranking-derived statistic (Kendall’s tau-b, Spearman’s rho, top- k overlap, pairwise reversal detection) for that specific condition and metric only – never scored as a loss or a tie, and never causing the whole condition to be dropped for policies that did complete. Every ranking-derived table reports its exclusion count (`n_conditions_excluded_for_undefined_metric`) alongside the statistic itself, since a source-concentrated exclusion rate is itself a reportable finding under RQ4, not a footnote.

5.7 Paired Design and Repetitions

Because the simulator is fully deterministic (§5.3), a single execution per (window, policy, region) cell is sufficient; ranking-uncertainty quantification (§6) instead comes from resampling over the independent workload windows themselves (block bootstrap), not from repeated stochastic runs of the same cell.

5.8 Mechanism Telemetry

Every executed cell additionally records a seven-field mechanism-activation telemetry summary: queue-depth (mean/max), batch-saturation (mean/max fraction of `max_active_sequences` in use), prefill/decode contention fraction, KV-occupancy (mean/max fraction of `max_kv_tokens` in use), admission-control activation count, preemption/reorder event count, and token-budget saturation fraction. Each field is always defined: a value of zero means either that the corresponding mechanism is genuinely absent for that policy (for example, preemption count is exactly zero for every policy that never issues a preempt/swap/migrate action, which is most of the panel) or that the underlying simulator mode does not implement a separate prefill/decode phase or token-budget model – not an instrumentation failure. This telemetry is collected for every cell but is not used to support any mechanism-level finding in this manuscript; RQ4’s telemetry-to-reversal association analysis (§6) is pre-specified and will only be run once Pilot-V2 outcomes exist.

5.9 Ranking-Uncertainty Quantification and Reversal Classification

Because the statistical unit of analysis is the workload window (§3.4), ranking-uncertainty is quantified by a whole-window block bootstrap: windows, not individual requests, are resampled with replacement (preserving each window’s internal request sequence intact), 2,000 resamples are drawn per condition, and every reported confidence interval is a 95% percentile interval over those resamples, using a deterministic, recorded seed so every interval is exactly reproducible from the frozen outcome matrix alone. This block structure exists because the requests within a window are not independent observations of scheduler behavior – they share one arrival process, one prompt/output-length mixture, and one operating region – so resampling at the request level would understate uncertainty; the window is the unit that is actually drawn independently by §4.3’s sampling design, and it is therefore the unit that must be resampled.

Two further tests calibrate the reversal statistics reported in §6.3 against multiple comparisons. A Friedman rank-sum omnibus test is applied per (source, region, metric) condition to ask whether the panel’s policies differ at all before any pairwise reversal is interpreted, so that pairwise reversal claims are read in the context of whether an omnibus difference exists in the first place. Because the panel and the (source, region, metric) grid together generate many pairwise comparisons, every pairwise reversal p -value is corrected across that condition’s family using Benjamini–Hochberg false-discovery-rate control at $q = 0.05$ before a reversal is classified as statistically supported; an uncorrected pairwise p -value alone is never sufficient for the `SUPPORTED_PRACTICAL_REVERSAL` class defined below.

A numerical sign change in a point estimate is not, by itself, a meaningful scheduler reversal from a systems standpoint: two policies can swap point-estimate order by a magnitude far smaller than the benchmark’s own measurement noise, in which case treating it as a reversal would overstate what the benchmark actually supports. Every pairwise (policy A, policy B) comparison at a given (source, region, metric) condition is therefore classified into exactly one of five preregistered classes, combining the practical-margin test with the bootstrap/FDR-corrected statistical test:

UNDEFINED_UNESTIMABLE At least one of the two policies’ metric values is undefined for this condition (§5.6); no direction is estimated or claimed.

STABLE_NO_SIGN_CHANGE The two policies’ relative order agrees across the compared conditions; no reversal occurred.

MICROSCOPIC_SIGN_CHANGE The point-estimate order reverses, but the winning margin never exceeds 10% of the losing policy’s value in either condition – a numerically real but practically

negligible sign flip, reported separately from, and never pooled with, the classes below.

UNSUPPORTED_SIGN_CHANGE_WIDE_CI The point-estimate order reverses with a practically meaningful margin in at least one condition, but the FDR-corrected 95% bootstrap interval for the difference does not exclude zero in both conditions – the apparent reversal is not statistically distinguishable from bootstrap noise at the preregistered confidence level.

SUPPORTED_PRACTICAL_REVERSAL The point-estimate order reverses *and* the winning margin exceeds 10% of the losing policy’s value in both compared conditions *and* the FDR-corrected 95% bootstrap interval for the direction excludes zero in both conditions. Only this class is reported as a headline reversal in §6.3; the other four classes are always reported alongside it, never omitted as a residual category.

5.10 Temporal Analysis Design

Temporal portability is evaluated separately per source, using each source’s own native chronology rather than a single shared calendar rule, because the three sources differ in what temporal structure they actually expose (§4.1). BurstGPT windows are assigned to EARLY/MIDDLE/ LATE chronology terciles by deterministic relative order within the trace (§4.3; 60/30/30 windows respectively), and a secondary EARLY/LATE bisect split is run as a robustness check on whether the tercile boundary choice itself drives any temporal finding (§5.11). Azure 2024’s temporal split instead uses the trace’s own absolute calendar timestamp, frozen at Unix epoch 1715731200.0 (2024-05-15T00:00:00Z), splitting the trace into before/after halves at that boundary; this is a genuine calendar split, not a relative-order tercile, because Azure’s release exposes an absolute timestamp field that BurstGPT’s schema does not (§4.2). Bailian/Qwen’s release does not expose a verified absolute timestamp field usable for a calendar split; its temporal analysis is therefore restricted to a relative-order comparison only, explicitly labeled **RELATIVE_CHRONOLOGY_ONLY** in every table it appears in, and is never presented alongside BurstGPT’s tercile result or Azure’s calendar-split result as though the three used an equivalent temporal definition (§6.4). No cross-source temporal number is computed by pooling these three distinct temporal definitions into one figure.

5.11 Robustness Analyses

Every headline ranking-portability result (§6) is pre-specified to be checked against a fixed set of robustness analyses: the 11-policy high-fidelity subset (**PRIMARY** panel excluding both **STYLE_APPROXIMATION** policies), leave-one-source-out re-estimation, window-size sensitivity, an alternate completion-conditioned-metric convention, the full six-region load grid, a temporal-split sensitivity check (§5.10’s BurstGPT bisect), and mechanism-family-exclusion checks. These robustness analyses are part of the frozen protocol; their results are reported alongside §6.7 once Pilot-V2 executes.

5.12 SLO-Definition Sensitivity: A Disclosed Protocol Gap

An SLO-definition sensitivity check – re-running the benchmark’s deadline-dependent policies under an alternative SLO-synthesis rule to confirm that any headline reversal is not an artifact of the one synthesis choice used to fabricate **slo_deadline** where a source does not natively carry it (§4.2) – was planned as part of this protocol. We disclose here, as a matter of methodological discipline rather than as a result, that no alternative SLO-synthesis rule was ever actually frozen before

or during Pilot-V2 execution: every document that references “the alternative SLO-synthesis rule” points to another document as the place it is defined, and that chain does not terminate in an actual formula, multiplier, or distribution anywhere in this project’s history. Consequently: no alternative rule was invented after Pilot-V2 execution began; no SLO-sensitivity experiment has been run; this robustness component is *not* claimed as preregistered evidence anywhere in this manuscript; and any future SLO-sensitivity study must be explicitly labeled a post-campaign robustness extension, with its own freeze and manifest process, unless a genuinely prior external protocol specifying the alternative rule is located.

6 Scheduler Ranking Portability

Every subsection below is a result contract: an exact specification of what will be reported once Pilot-V2 executes (§11 states current execution status), reproduced from `docs/MANUSCRIPT_RESULT_CONTRACT.md`, the project’s single source of truth for contract wording. No subsection below asserts a direction of finding. Either a high-portability or a low-portability outcome is a complete, reportable result under this design (§9).

6.1 Discriminability

[PENDING PILOT-V2 RESULT: fraction of non-tied (window, load-region) conditions per source, per metric, over the full 13-policy / 6-region Pilot-V2 matrix, with Wilson 95% confidence intervals per source (method reused from the Stage-0 diagnostic, §5.4). Historical Stage-0 evidence (3 sources $\times$ 3 regions $\times$ 6-policy subset only) is reported separately in §5.4 and is not substituted for this result.]



Figure 1: Placeholder: source-pair $\times$ load-region ranking-portability heatmap (§6.2). Cell color/value is Kendall’s tau-b between the two sources’ policy rankings at that operating region, with the block-bootstrap 95% interval reported in the underlying table; cells with an excluded-condition count above a reported threshold are marked, not silently colored as if fully defined (§5.6).

6.2 Cross-Source Rankings

[PENDING PILOT-V2 RESULT: Kendall’s tau-b and Spearman’s rho between each pair of the three sources’ policy rankings, per metric (§5.6), per operating region, with block-bootstrap 95% confidence intervals over windows. Reported alongside top-1/top-3 overlap and `n_conditions_excluded_for_undefined_metric` per cell. No claim of “stable” or “unstable” rankings is made before this table exists.]

6.3 Pairwise Reversals

[PENDING PILOT-V2 RESULT: every (policy A, policy B) pair’s count in each of the five reversal classes of §5.9 (`UNDEFINED_UNESTIMABLE`, `STABLE_NO_SIGN_CHANGE`, `MICROSCOPIC_SIGN_CHANGE`, `UNSUPPORTED_SIGN_CHANGE_WIDE_CI`, `SUPPORTED_PRACTICAL_REVERSAL`), reported sepa-

rately – never pooled into a single “reversal count” that mixes microscopic and supported-practical classes.]

[REVERSAL-CLASS MATRIX]	
(policy A, policy B) × reversal class, counts per class	

Figure 2: Placeholder: reversal-class matrix (§6.3). Rows are policy pairs, columns are the five classes of §5.9; only the `SUPPORTED_PRACTICAL_REVERSAL` column is reported as a headline finding, with the other four columns always shown alongside it.

6.4 Temporal/OOD Portability

[PENDING PILOT-V2 RESULT: three separately labeled comparisons – within-BurstGPT EARLY/MIDDLE/LATE tau, Azure-2024 own-window calendar-split tau, and cross-source (provider/domain) tau from §6.2 – never merged into a single “temporal/domain OOD” number.]

6.5 Load Conditioning

Load-level dependence is a secondary robustness/sensitivity analysis (§3.1), not a headline claim of cross-source rank instability by itself.

[PENDING PILOT-V2 RESULT: tau and reversal frequency across the six frozen operating regions (§5.5), per source, reported as a robustness/sensitivity table using this project’s own independently calibrated load protocol – never LLM 2026’s {1, 2, 4, 8, 16, 32, 64, 128} grid or its 60-window set (§3.3).]

6.6 Metric Conditioning

[PENDING PILOT-V2 RESULT: tau between every pair of metrics in the metric contract (§5.6), per source, reported alongside whether any exclusion-driven gap from undefined completion-conditioned metrics is concentrated in one source – a candidate explanatory factor if so, reported descriptively, not causally.]

6.7 Robustness

[PENDING PILOT-V2 RESULT: sensitivity of §6.1 through §6.6’s headline statistics to the 11-policy high-fidelity subset, leave-one-source-out re-estimation, window-size choice, the alternate completion-conditioned-metric convention, the full six-region load grid, a temporal-split sensitivity check, and mechanism-family-exclusion checks (§5.11).]

7 Benchmark Sample Complexity

7.1 Motivation

Every scheduler-ranking result in this manuscript is itself derived from a finite window budget – 40 windows per source, 120 total (§4.3). A systems researcher designing a future scheduler comparison faces a practical, resource-bound version of the same question this benchmark studies at the source/region/metric level: *if only n workload windows can be afforded, how reliably would the*

resulting scheduler-ordering conclusion reproduce the conclusion obtained from the full window collection? This question is itself a systems-evaluation parameter, not a purely statistical afterthought: it bounds the experimental cost of a trustworthy scheduler comparison, it affects how much confidence a reader should place in a paper that evaluates on a handful of trace windows, and it is a precondition for treating this benchmark’s own frozen 120-window corpus (rather than some larger or smaller corpus) as an adequate evidentiary base for the results in §6.

7.2 Design

[PENDING RESULT: probability of recovering the full $n = 40$ -window ranking at $n \in \{5, 10, 20, 30, 40\}$ windows per source, using at least 500 draws per n (sampling without replacement from the full window pool), per source and per metric, including a comparison between a concentrated (single-source-heavy) and a distributed (evenly-spread) window budget. Recovery is reported both as exact-order recovery and as top- k ($k \in \{1, 3\}$) recovery relative to the full-window reference ranking. We report the smallest n at which recovery probability first exceeds the preregistered 0.9 threshold; no specific value of n is claimed before this analysis exists.]

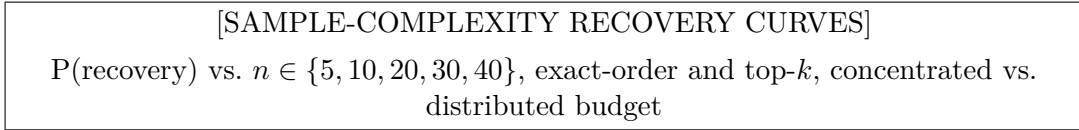


Figure 3: Placeholder: benchmark sample-complexity recovery curves (§7.2). One curve per (recovery type, window-budget strategy) pair, with the preregistered 0.9 recovery threshold marked; no specific n value is asserted until the underlying analysis runs.

This analysis directly answers RQ5 (§3.1) and depends on the full Pilot-V2 outcome matrix (§6); it is not run on Stage-0’s historical, smaller pilot (§5.4).

8 Real-System Validation

8.1 Objective

This analysis answers RQ6 (§3.1). We do not claim, here or anywhere in this manuscript, that simulated absolute latency or throughput values match real-hardware values; simulator latency is not assumed to transfer to real-engine latency, and the simulator’s own load-calibration reference λ_{ref} (§5.5) is not assumed to transfer to a real engine’s saturation point either. The only claim in scope is agreement in *relative ranking* and *reversal direction* between the simulator and a real vLLM deployment on a small, frozen set of validation cases – not a hardware duplication of the full 18,720-cell Pilot-V2 matrix (docs/REAL_SYSTEM_VALIDATION_PLAN.md).

8.2 Planned Design

The following design is prepared, and its supporting harness – server/ client request orchestration against a real vLLM deployment, randomized/ ABBA execution ordering, and per-run provenance capture – has been functionally smoke-tested (§11); it is not yet run as a scientific validation, because case selection depends on §6.2/§6.3, which do not yet exist.

- **Mechanisms.** Approximately four representative scheduling mechanisms, one from each stratum in §5.2 that has a real or faithfully reimplementable execution path on an actual

serving engine – e.g. `fifo`, `vllm_faithful` (or its chunked-prefill successor), and `sarathi_faithful`, plus a fairness-aware mechanism where a real-engine analogue can be established; `apt_serve_faithful` is excluded, consistent with its `scaffolding_only` status (§5.1).

- **Cases.** Three to four workload/case conditions, chosen to include at least one case exhibiting a preregistered `SUPPORTED_PRACTICAL_REVERSAL` (§5.9) from the simulated analysis and at least one stable-ordering control condition – selection frozen only from this study’s own completed results, never from a partial or in-progress result.
- **Operating regions.** `PRE_KNEE` and `KNEE/OVERLOAD`, recalibrated against the real engine’s own observed saturation point rather than reusing the simulator’s calibrated load factors (§5.5) directly.
- **Repetitions.** At least five repetitions per (mechanism, case, region) cell, in contrast to the simulator’s deterministic single execution plus rep0/rep1 verification (§5.7) – real hardware execution is not deterministic, so repeated stochastic trials are required here even though they were not required in simulation.
- **Ordering.** Randomized or ABBA ordering of mechanism/cell execution, to avoid confounding a mechanism’s measured performance with time-of-day GPU-sharing effects.
- **Manifests.** Fixed workload request manifests, generated once per case and reused verbatim across mechanisms and repetitions, with an explicit warmup phase discarded before measurement.
- **Provenance.** Every real-engine run records its exact hardware model, driver and CUDA version, Python/PyTorch/vLLM version, model identity and revision, scheduler configuration, workload-manifest hash, run-order seed, and repository commit SHA, so that any released real-validation row is independently reproducible (§11).

8.3 Result Placeholder

[PENDING RESULT: sign agreement, Kendall’s tau, and reversal-agreement between simulated and real-vLLM policy rankings, for the frozen validation cases above. Case selection must be frozen from this study’s own §6.2/§6.3 results before any real-vLLM run is executed, and cannot be written until those results exist.]

[REAL VALIDATION HARDWARE]
[REAL VALIDATION CASES]
[REAL/SIM RANK AGREEMENT]
[REAL VALIDATION RESULT]

Figure 4: Placeholder: simulator-vs-real-vLLM ranking agreement for the frozen validation cases (§8.2), reported as sign agreement and Kendall’s tau per case, never as an absolute latency/throughput comparison. Populated only after case selection is frozen from §6.3.

9 Discussion

Because §6–§8 report no comparative outcome yet, we structure this discussion around both directions the pending result may take, so that the discussion below requires no rewriting once the outcome is known – only the removal of whichever branch did not occur.

9.1 If Ranking Portability Is High

If scheduler rankings prove largely portable across the three independently sourced workload families, operating regions, and metrics tested (§6.2–§6.6), the practical implication is that future scheduler comparisons may not need to be repeated across many independently sourced workloads to be trustworthy: a smaller, well-chosen evaluation footprint could suffice, reducing the evidence burden of scheduler evaluation. A high-portability result would also constitute evidence of a kind of robustness for the panel’s mechanisms themselves – that their relative strengths and weaknesses are not artifacts of any one workload’s specific structure. At the same time, a high-portability finding on this benchmark’s three sources and fixed operating-region grid would not license an unconditional claim of portability to every workload a practitioner might encounter; the boundary conditions under which portability was observed (source identity, load range, metric set, and simulator fidelity; §10) would need to be stated precisely, and §7’s result would still bound how much evidence is required before a specific portability claim is statistically warranted.

9.2 If Ranking Portability Is Low

If scheduler rankings instead prove fragile – with practically meaningful, statistically supported reversals concentrated in particular sources, regions, or metrics (§6.3) – the practical implication is the opposite: single-workload scheduler evaluations risk over-generalizing, and a comparative claim of the form “scheduler *A* outperforms scheduler *B*” should be read as conditional on the workload, metric, and load region it was measured under, unless independently re-verified. This would motivate multi-source evaluation as a de facto requirement for general-purpose scheduler comparisons going forward, and would sharpen the importance of §7’s sample-complexity result for practitioners who cannot afford to test every candidate workload distribution. A low-portability finding would also raise the practical question of which workload characteristics predict a reversal (RQ4, §4.5, §5.8) – explanatory, not selector-building, in scope (§3.3).

9.3 Mechanism Equivalence and Collapse

Independent of which branch above applies at the ranking-portability level, the mechanism telemetry collected for every cell (§5.8) allows a finer-grained discussion of *why* rankings behave as observed – for example, whether policies that appear to tie on a given workload do so because their scheduling decisions genuinely converge (as Stage 0’s historical diagnostic found for several policies on BurstGPT’s structurally short-prompt traffic; §5.4) or because the metric used cannot distinguish otherwise-different decision sequences. This discussion is deferred to the post-Pilot-V2 revision of this section, once §6.2–§6.6 exist to anchor it.

9.4 Workload Descriptor Interpretation

The workload-separability result (§4.6) establishes that the three sources are observably different, and that prompt-length structure is the primary driver of that separability (§4.7). Whatever RQ4’s descriptor-to-reversal association analysis finds, it should be read against this backdrop: an

association between a descriptor and a reversal does not by itself establish that the descriptor is a genuine *causal* driver of the reversal, only that it is a source-separating feature that co-occurs with where reversals were or were not observed.

9.5 Benchmark Construction and Practical Reproducibility

Regardless of outcome direction, this study’s paired design, fixed scheduler panel, and preregistered protocol (§3) are intended to make the benchmark itself reusable: a future scheduler design can be dropped into the same panel and evaluated against the same frozen windows, operating regions, and metric contract, using the planned public artifact (§11) rather than re-deriving the protocol from this manuscript’s prose.

10 Threats to Validity

Simulator abstraction. All results in this manuscript are produced by a discrete-event simulator, not a real serving engine (§5.3). We do not claim that simulated absolute latency or throughput values match real-hardware values; only relative ranking and reversal direction are claimed to transfer, and only for the frozen validation cases in §8, once that section’s result exists.

Fidelity of reimplemented policies. FAITHFUL_EXTERNAL policies are validated reimplementations, not the original authors’ code; residual implementation gaps relative to the original published systems are possible despite unit-test validation (§5.2).

STYLE_APPROXIMATION baselines. The two style-level approximation policies are explicitly excluded from the PRIMARY headline panel and used only for a robustness check; they must not be read as official or validated reimplementations of their namesake systems (§5.2).

Three primary workload families. The Pilot-V2 corpus draws from three primary sources (§4.1); a fourth candidate source (TraceLab) is retained only as out-of-distribution context because its independence from a prior public window sweep cannot be fully verified. Any portability finding in §6 is scoped to these three sources plus whatever out-of-distribution evidence TraceLab can responsibly contribute, not to LLM-serving workloads in general.

Dataset/provider dependence. As established in §4.1, BurstGPT and this project’s Azure traces are not established to come from independent underlying providers – both trace back to Microsoft Azure infrastructure, collected through different mechanisms. Any cross-source finding involving BurstGPT and Azure should be read with this in mind; Bailian/Qwen remains a genuinely independent platform.

Prompt-token semantic differences. Prompt-token fields compose differently across sources (single-request figure for BurstGPT versus potentially multi-turn-cumulative context for Azure and TraceLab; §4.4). Any prompt-length-based claim in this manuscript carries this caveat and should not be read as evidence that the sources’ underlying conversations differ in length, only that their observed, request-level prompt-token figures do.

Unresolved historical byte-level overlap. An exact, byte-level reconstruction of a prior related project’s specific 60-window public-trace replay set could not be independently re-derived from this project’s available checkout; that project’s own recorded parameters and final verdict are taken on the strength of direct manuscript inspection, not independently re-derived from local artifacts. This project’s own window construction is independently verified as non-identical (§4.3, §3.3), but the historical byte-level reconstruction gap itself remains an open item.

Homogeneous hardware / execution assumptions. The simulator executes every cell under a fixed, single-GPU-located execution model for the PRIMARY panel; two disaggregation-requiring policies are confined to a secondary stratum and never pooled with the PRIMARY leaderboard (§5.1).

Completion-conditioned undefined metrics. Several metrics are undefined, not zero, when a cell records zero completions (§5.6); this is a real methodological choice with a real cost – an undefined-metric policy is excluded from that specific condition’s ranking-derived statistics, which lowers effective sample size for those conditions and must be read alongside every ranking-derived table’s reported exclusion count.

Phase-11 zero-completion prevalence. 301 of 720 FIFO-only calibration-region records recorded zero completions (§5.5). This is disclosed as a calibration characteristic, not a headline finding; it does, however, mean that a non-trivial share of (window, region) combinations will contribute no completion-conditioned-metric observation to §6’s downstream analyses for any policy at that specific region, which is a structural constraint on statistical power in the highest-pressure regions in particular.

SLO-definition sensitivity is disclosed, not claimed. An SLO-synthesis sensitivity check was planned but no alternative SLO-synthesis rule was ever actually preregistered before or during Pilot-V2 execution (§5.12); no alternative rule was invented after the fact, no such experiment has been run, and this robustness component is not claimed as evidence anywhere in this manuscript. The mitigation is disclosure and process, not substitution: any future SLO-sensitivity study will be run as an explicitly labeled post-campaign extension with its own frozen protocol, never retrofitted into this study’s preregistered evidence.

Scope of real-system validation. §8’s validation is limited by design to a small, frozen set of case-selected conditions (largest-effect-size reversal plus one stable-ordering control), not a full re-execution of the Pilot-V2 matrix on real hardware; agreement or disagreement on those specific cases should not be over-generalized to every cell in §6.

11 Reproducibility and the LSSP Artifact

We plan to release the LLM-Serving Scheduler Portability Benchmark (LSSP) as a public artifact at the Hugging Face repository slug `SoroushVahidi/llm-serving-scheduler-portability`. **As of this manuscript, that artifact is planned but not yet released.** The intended release will include: the frozen workload window identities and their provenance (§4.3); the scheduler panel and its configuration (§5.1); the six-region calibration protocol and its outputs (§5.5); the full set of paired policy outcomes across windows, regions, and metrics, once generated (§6); the mechanism telemetry collected for every cell (§5.8); the ranking-portability analysis outputs

(Kendall/Spearman statistics, reversal tables, block-bootstrap confidence intervals); and the reproduction scripts used to regenerate every table and figure in this manuscript from those frozen inputs.

Every generated artifact in the planned release is designed to carry, or link via a provenance manifest, its generating repository commit SHA, resolved experiment-config hash, workload-manifest hash, policy-registry version, dataset-source checksums, environment version, and (where relevant) per-cell seed, so that any released row can be regenerated from this project’s own commit history alone rather than depending on an external, possibly-rewritten historical commit.

The immutable scientific identifiers already fixed at the time of this manuscript are recorded in Table 1 and in `docs/ARTIFACT_HASH_LEDGER.md`: the Phase-10 workload-window freeze content hash, the Phase-10 compact index hash, the Phase-11 prelaunch freeze contract hash, the Phase-11 raw FIFO calibration output hash, and the Phase-11 region-assignment output hash. These five identifiers were independently re-verified as unchanged, and consistent across every cross-referencing project document, immediately before this manuscript’s authoritative branch was committed (§12).

12 Conclusion

We have formulated the portability of a comparative scheduler ranking, $R(W, M, L)$, as a benchmark object distinct from both workload characterization and single-scheduler evaluation, and we have designed, frozen, and documented a paired, multi-source evaluation protocol – a mechanism-diverse fixed scheduler panel, a calibrated six-region operating grid, an explicit metric contract, and an uncertainty/reversal-detection methodology – before generating any comparative outcome. We have established, as project-local evidence available now, that the three primary workload sources underlying this benchmark are measurably different and strongly separable in their observed descriptor distributions (§4), and, as historical pilot evidence, that scheduler differentiation under this project’s protocol is real but strongly source-dependent, motivating the larger panel and denser load grid used going forward (§5.4). The benchmark’s central questions – whether comparative scheduler rankings are portable across independent sources, operating regions, and metrics, how many workload windows are required before such a ranking is trustworthy, and whether simulated rankings reproduce on a real serving engine – remain open at the time of this manuscript and are stated as explicit result contracts (§6–§8) rather than asserted in either direction. We plan to release LSSP as a public, independently reproducible artifact (§11) once these results exist.

Declarations

Funding. The author declares that no funds, grants, or other support were received during the preparation of this manuscript. *[AUTHOR TO CONFIRM before submission.]*

Competing interests. The author declares no competing interests. *[AUTHOR TO CONFIRM before submission.]*

Author contributions. This is a sole-authored manuscript; Soroush Vahidi is responsible for study design, implementation, analysis, and writing.

Ethics approval. Not applicable. This study evaluates scheduling-policy performance on request-level traffic traces (arrival times, token counts) and does not involve human subjects, personal data, or content requiring ethics review.

Data availability. This study distinguishes four categories of data, consistent with §11:

- **Externally sourced public traces** (BurstGPT, Azure 2024, Bailian/Qwen) are governed by their original publishers’ release terms and license status (§4.2; BurstGPT: CC-BY-4.0) and are not redistributed as part of this project’s own release; readers should obtain them from their original sources.
- **Derived benchmark manifests** – the frozen window identities, provenance, and content hashes (§4.3) – and
- **Generated campaign results** – the full paired policy outcome matrix across windows, regions, and metrics (§6) – and
- **Analysis artifacts** – ranking-portability statistics, reversal tables, and reproduction scripts (§11) – will be archived at a persistent, publicly accessible repository (the Hugging Face repository slug `SoroushVahidi/llm-serving-scheduler-portability`) upon completion of the study reported here. **As of this manuscript, that artifact is planned but not yet released** (§11); no data availability claim in this statement should be read as an assertion that the release is already live.

Code availability. The simulator, policy implementations, and analysis pipeline underlying this study will be released alongside the data described above, under the same repository, once the study completes.

Table 1: Immutable scientific identifiers, re-verified unchanged immediately before this manuscript’s authoritative branch was committed (`docs/ARTIFACT_HASH_LEDGER.md`).

Artifact	SHA-256
Phase-10 scientific window (content hash)	0d1aa06ccbee352207327ea369ae75f12e91c0cda006c813a41b381effd29eef
Phase-10 compact index	d78ec1087fedae02174ca093a9860c70468be336ccb1d7e6de756c81ba331e53
Phase-11 prelaunch freeze contract	e2564ea9484190832de50f63173c4b73ae054d6ae7008bb4ff6648c8dc917f7b
Phase-11 raw FIFO calibration	201caaf04476ad8737ef6079fc0d6cb4e864601711d0b96c88750a717d8b2a6a
Phase-11 region-assignment output	9fcb92f9ea1206ce185194527ada35d0e3b91bf4904be7ae23ba9ea997c17574

References

- [1] Amey Agrawal, Nitin Kedia, Nipun Kwatra, Nikhil Sarda, Alexey Tumanov, et al. Sarathiserve: Efficient llm serving with chunked prefills and continuous batching. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2403.00050>.
- [2] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for llm inference. In *Proceedings of Machine Learning and Systems (MLSys ’24)*. MLSys, 2024. URL <https://arxiv.org/abs/2405.05465>.

- [3] anonymized per venue record; see primary source. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)*, 2025. URL <https://arxiv.org/abs/2506.02634>. Characterizes KV-cache reuse patterns at a large cloud provider (Alibaba Cloud); workload-characterization context only.
- [4] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. Optimal scheduling algorithms for llm inference: Theory and practice. *ACM Measurement and Analysis of Computing Systems*, 2025. URL <https://arxiv.org/abs/2508.01002>.
- [5] Alejandro Carmona-Martínez, Gregorio Bernabé, and José M. García. Characterization of machine learning compilers for llm inference on nvidia gpus. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08559-6.
- [6] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, et al. Llm-servingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale. In *Proceedings of the 2024 IEEE International Conference on Computer Design (ICCD '24)*. IEEE, 2024. URL <https://arxiv.org/abs/2408.05499>.
- [7] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. Saia: a seamless slurm-native solution for hpc-based services. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08508-3.
- [8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*, 2024. URL <https://arxiv.org/abs/2408.15792>.
- [9] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. S³: Increasing gpu utilization during generative inference for higher throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS '23)*, 2023. doi: 10.5555/3666122.3666913. URL <https://arxiv.org/abs/2306.06000>. Distinct from efficientllmscheduling2024 (NeurIPS '24). S³ predicts output-length to pack KV-cache memory at admission time; it does not define a learning-to-rank decode-order policy.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. vllm: Easy, fast, and cheap llm serving with pagedattention. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*. USENIX Association, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [11] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE/ACM, 2024. doi: 10.1109/ISCA59077.2024.00019. URL <https://arxiv.org/abs/2311.18677>. Primary analysis paper and release mechanism for AzureLLMInference-Dataset2023 (conversation and code splits, collected 2023-11-11), used by this project as azure2023trace.
- [12] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a kvcache-centric architecture for serving llm chatbot. In *Proceedings of the 23rd USENIX*

- Conference on File and Storage Technologies (FAST '25)*. USENIX Association, 2025. URL <https://arxiv.org/abs/2407.00079>.
- [13] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic llm workloads. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL <https://www.usenix.org/conference/nsdi26/presentation/ruan-libra>.
 - [14] see primary source for full author list. Apt-serve: Adaptive request scheduling on hybrid cache for scalable llm inference serving. *Proceedings of the ACM on Management of Data (PACMMOD)*, 2025. URL <https://arxiv.org/abs/2504.07494>.
 - [15] see primary source for full author list. Llmserveingsim2.0: A unified simulator for heterogeneous hardware and serving techniques in llm infrastructure. *IEEE Computer Architecture Letters*, 24(2), 2025. URL <https://arxiv.org/abs/2511.07229>. An expanded companion version circulates as arXiv:2602.23036 ("...Heterogeneous and Disaggregated LLM Serving Infrastructure"); treat as the same simulator lineage as llmservingsim2024.
 - [16] see primary source for full author list. Proserve: Unified multi-priority request scheduling for llm serving. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2512.12928>.
 - [17] see primary source for full author list. A year in llm serving: Workload evolution, caching and load-balancing. *arXiv preprint*, 2026. URL <https://arxiv.org/abs/2608.13573>. Peer review status unverified: very recent arXiv preprint (2026-07), one-year production trace from the Chutes platform. Cite only as recent contextual background, not as an established result.
 - [18] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024. URL <https://arxiv.org/abs/2401.00588>. Introduces the Virtual Token Counter (VTC) fair scheduling algorithm.
 - [19] Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Hao Zhang, et al. Llumnix: Dynamic scheduling for large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*, 2024. URL <https://arxiv.org/>.
 - [20] Patricia Siwinska, Jie Lei, Adrián Castelló, Pedro Alonso-Jordá, and Enrique S. Quintana-Ortí. Enhancing transformer performance and portability through auto-tuning frameworks. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08327-6.
 - [21] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*. IEEE, 2025. URL <https://arxiv.org/abs/2408.00741>. Primary analysis paper and release mechanism for AzureLLMInferenceDataset2024, used by this project as azure2024trace.
 - [22] Yinghao Tang, Tingfeng Lan, Xiuqi Huang, Hui Lu, and Wei Chen. Scorpio: Serving the right requests at the right time for heterogeneous slos in llm inference. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2505.23022>.

- [23] Yiheng Tao, Yihe Zhang, Matthew T. Dearing, Xin Wang, Yuping Fan, and Zhiling Lan. Pars: Low-latency llm serving via pairwise learning-to-rank. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2510.03243>.
- [24] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize llm serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*, Volume 2. ACM, 2025. doi: 10.1145/3711896.3737413. URL <https://arxiv.org/abs/2401.17644>. Final published version. arXiv:2401.17644 first released 2024-01-31 (v1); the KDD 2025 camera-ready corresponds to arXiv v4/v5 (2025-05-21 / 2025-05-26). Dataset release: github.com/HPMLL/BurstGPT, tag v2.0.
- [25] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fastserve: Iteration-level preemptive scheduling for large language model inference. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL <https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang>.
- [26] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. Servegen: Workload characterization and generation of large language model serving in production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL <https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen>.
- [27] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 2022. URL <https://arxiv.org/abs/2207.00032>.
- [28] Netanel Zakay and Dror G. Feitelson. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience*, 26(12): 2079–2105, 2014. doi: 10.1002/cpe.3240.
- [29] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. Simple is better: Multiplication may be all you need for llm request scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association, 2026. URL <https://arxiv.org/abs/2603.15202>.
- [30] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. Jitservice: Slo-aware llm serving with imprecise request information. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL <https://www.usenix.org/conference/nsdi26/presentation/zhang-wei>.
- [31] Yinmin Zhong, Shengyu Liu, Junda Chen, Bingyang Wu, Chen Chen, Zili Zhang, Yibo Zhu, Xin Jin, and Hao Zhang. Distserve: Disaggregated prefill and decode for goodput-optimized llm serving. In *Proceedings of the 2024 USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, 2024. URL <https://arxiv.org/abs/2401.00000>.

- [32] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Tracelab: Characterizing coding agent workloads for llm serving. *arXiv preprint*, 2026. URL <https://arxiv.org/abs/2606.30560>.